

ASSESSMENT TOOL 3**COURSE NAME: Database Management****Systems COURSE CODE: CSA0501**

COURSE OUTCOME COVERED

C02: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)

~~Assessment Tool Weightage: 30%~~

Dr

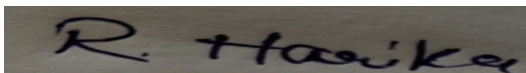
QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.	BL3 – Apply	5
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.	BL3 – Apply	5
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.	BL3 – Apply	5
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation.	BL3 – Apply	5
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.	BL3 – Apply	5
6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.	BL6 – Create	5
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.	BL3 – Apply	5

8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.	BL3 – Apply	5
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.	BL6 – Create	5
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.	BL3 – Apply	5

Code of Conduct:

I R.HARIKA(192572140) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained

1. Incorrect JOIN Syntax Returning Duplicate Results

Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.

Incorrect Query

```
SELECT E.EmpName, D.DeptName  
FROM EMPLOYEE E, DEPARTMENT D;
```

Error

- The query uses **Cartesian Product (Cross Join)** because there is no JOIN condition.
- Every employee is matched with every department, resulting in duplicate/unwanted rows.

Correct Query

```
SELECT E.EmpName, D.DeptName  
FROM EMPLOYEE E  
INNER JOIN DEPARTMENT  
ON E.DeptID = D.DeptID;
```

Output

EmpName DeptName

Rahul	HR
Priya	IT
Arjun	HR

Explanation

Using INNER JOIN with ON E.DeptID = D.DeptID returns only matching employee-department records.

2. Debug Correlated Sub-query

The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.

Incorrect Query

```
SELECT EmpName  
FROM EMPLOYEE  
WHERE Salary >
```

```
(  
SELECT AVG(Salary)  
FROM EMPLOYEE  
);
```

Error

- Compares employee salary with the **overall company average** instead of the **department average**.
- Missing correlation with the outer query.

Correct Query

```
SELECT EmpName  
FROM EMPLOYEE E  
WHERE Salary >  
(  
SELECT AVG(Salary)  
FROM EMPLOYEE  
WHERE DeptID = E.DeptID  
);
```

Output

EmpName

Arjun

Priya

Explanation

The sub-query is correlated using:

```
WHERE DeptID = E.DeptID
```

This compares each employee with the average salary of **their own department**.

3. Debug SQL View

A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.

Incorrect View

```
CREATE VIEW Employee_View AS  
  
SELECT *  
  
FROM EMPLOYEE  
  
GROUP BY DeptID;
```

Error

- SELECT * cannot be used with GROUP BY unless every non-aggregated column appears in the GROUP BY clause.
- Causes SQL error.

Correct View

```
CREATE VIEW Employee_View AS  
  
SELECT DeptID,  
       AVG(Salary) AS AvgSalary  
  
FROM EMPLOYEE  
  
GROUP BY DeptID;
```

Output

DeptID AvgSalary

D1	52500
D2	52500

Explanation

The corrected view groups by DeptID and calculates the average salary correctly.

4. Debug Relational Algebra Expression

Debug the given relational algebra expression that produces incorrect output for a Natural Join operation.

Incorrect Expression

$\text{EMPLOYEE} \times \text{DEPARTMENT}$

Error

- Uses **Cartesian Product** instead of **Natural Join**.

- Produces unnecessary combinations.

Correct Expression

EMPLOYEE $\bowtie$ DEPARTMENT

or

EMPLOYEE $\bowtie$ EMPLOYEE.DeptID = DEPARTMENT.DeptID DEPARTMENT

Output

EmpID EmpName DeptName

101 Rahul HR

102 Priya IT

103 Arjun HR

Explanation

Natural Join combines only matching department records.

5. Referential Integrity Violation

The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.

Incorrect Query

INSERT INTO EMPLOYEE

VALUES

(101,'Rahul',50000,50);

(Department 50 does not exist.)

Error

- DeptID 50 is absent in the DEPARTMENT table.
- Violates the Foreign Key constraint.

Correct SQL

CREATE TABLE DEPARTMENT

(

DeptID INT PRIMARY KEY,

```
DeptName VARCHAR(30)
```

```
);
```

```
CREATE TABLE EMPLOYEE
```

```
(
```

```
EmpID INT PRIMARY KEY,
```

```
EmpName VARCHAR(50),
```

```
Salary INT,
```

```
DeptID INT,
```

```
FOREIGN KEY(DeptID)
```

```
REFERENCES DEPARTMENT(DeptID)
```

```
);
```

```
INSERT INTO DEPARTMENT
```

```
VALUES
```

```
(50,'HR');
```

```
INSERT INTO EMPLOYEE
```

```
VALUES
```

```
(101,'Rahul',50000,50);
```

Output

1 row inserted into DEPARTMENT.

1 row inserted into EMPLOYEE.

Explanation

The parent record is inserted before the child record, preserving referential integrity.

6. Optimize Nested Query Using JOIN

Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.

Inefficient Query

```
SELECT EmpName
FROM EMPLOYEE
WHERE DeptID IN
(
SELECT DeptID
FROM DEPARTMENT
WHERE DeptName='HR'
);
```

Optimized Query

```
SELECT E.EmpName
FROM EMPLOYEE E
INNER JOIN DEPARTMENT D
ON E.DeptID = D.DeptID
WHERE D.DeptName='HR';
```

Output

EmpName

Rahul

Arjun

Explanation

Using JOIN avoids unnecessary nested sub-query execution and improves performance.

7. Debug GROUP BY Query

The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.

Incorrect Query

```
SELECT DeptID,
```



```
EmpName,  
SUM(Salary)  
FROM EMPLOYEE  
GROUP BY DeptID;
```

Error

- EmpName is neither grouped nor aggregated.
- SQL returns an error or incorrect result.

Correct Query

```
SELECT DeptID,  
SUM(Salary) AS TotalSalary  
FROM EMPLOYEE  
GROUP BY DeptID;
```

Output

DeptID TotalSalary

D1	105000
D2	105000

Explanation

All non-aggregated columns in the SELECT clause must appear in the GROUP BY clause. EmpName is removed because it is neither grouped nor aggregated.

8)Identify and Fix the Logical Error in a SQL UPDATE Statement

Question:

Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.

EMPLOYEE Table

EmpID EmpName Salary

101	Rahul	50000
-----	-------	-------

EmpID EmpName Salary

102 Priya 60000

103 Arjun 55000

Incorrect SQL Query

UPDATE EMPLOYEE

SET Salary = Salary + 5000;

Error

- The UPDATE statement has **no WHERE clause**.
- As a result, **all employee salaries** are increased by ₹5,000, which may not be intended.

Correct SQL Query

Suppose only employee **EmpID = 101** should receive the increment.

UPDATE EMPLOYEE

SET Salary = Salary + 5000

WHERE EmpID = 101;

Output**Before Update****EmpID EmpName Salary**

101 Rahul 50000

102 Priya 60000

103 Arjun 55000

After Update**EmpID EmpName Salary**

101 Rahul 55000

102 Priya 60000

EmpID EmpName Salary

103 Arjun 55000

Explanation

- The **WHERE** clause restricts the update to the intended row.
- Without the WHERE clause, all rows in the table are modified.

9) Optimize Multi-table SQL Query

Question:

Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.

EMPLOYEE

EmpID EmpName DeptID

101 Rahul D1

102 Priya D2

103 Arjun D1

DEPARTMENT

DeptID DeptName

D1 HR

D2 IT

Inefficient Query

SELECT EmpName

FROM EMPLOYEE

WHERE DeptID IN

(

SELECT DeptID

```
FROM DEPARTMENT  
WHERE DeptName='HR'  
);
```

Problem

- Uses a sub-query that may be executed repeatedly.
- Less efficient for large databases.

Optimized Query Using JOIN

```
SELECT E.EmpName  
FROM EMPLOYEE E  
INNER JOIN DEPARTMENT D  
ON E.DeptID = D.DeptID  
WHERE D.DeptName = 'HR';
```

Recommended Indexes

```
CREATE INDEX idx_emp_dept  
ON EMPLOYEE(DeptID);
```

```
CREATE INDEX idx_dept_name  
ON DEPARTMENT(DeptName);
```

Output

EmpName

Rahul

Arjun

Explanation

- INNER JOIN retrieves matching records more efficiently than nested sub-queries.

- Indexes on DeptID and DeptName improve search and join performance.

10) Debug Relational Calculus Expression

Question:

Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.

EMPLOYEE Relation

EmpID EmpName DeptID

101	Rahul	D1
102	Priya	D2
103	Arjun	D1

Incorrect Tuple Relational Calculus Expression

```
{ E.EmpName |
  EMPLOYEE(E) ∧
  ∀ D (D.DeptID = E.DeptID ∧ D.DeptName = 'HR')
}
```

Error

- The **universal quantifier (∀)** incorrectly requires **every department** to satisfy DeptName = 'HR'.
- Since departments such as IT and Finance also exist, the condition is false for all employees.
- Therefore, the result is an **empty set**.

Correct Tuple Relational Calculus Expression

```
{ E.EmpName |
  EMPLOYEE(E) ∧
  ∃ D (DEPARTMENT(D) ∧
    D.DeptID = E.DeptID ∧
```

```
D.DeptName = 'HR')  
}
```

Output

EmpName

Rahul

Arjun

Explanation

- Replace the universal quantifier ($\forall$) with the existential quantifier ($\exists$).
- The corrected expression returns employees who belong to **at least one HR department**, rather than requiring all departments to be HR.

